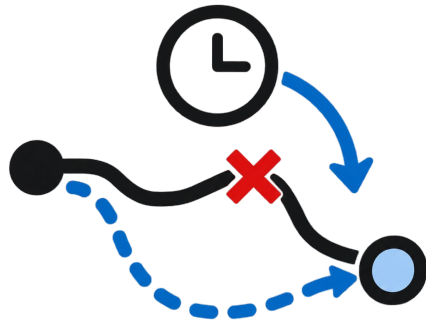


A Deterministic Asynchronous Service-Based Architecture for Agent-Based Transport Simulation

Paul Heinrich, Kai Nagel

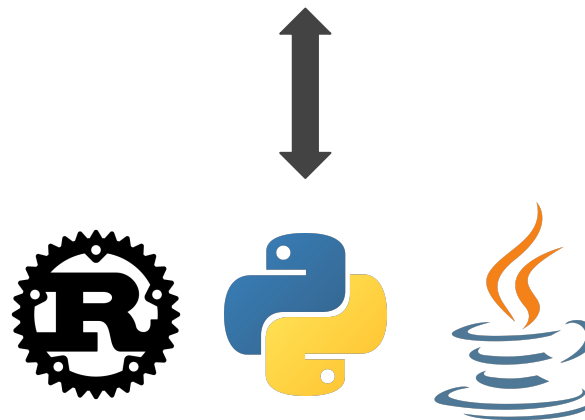
Motivation

(1) Speed up within-day replanning

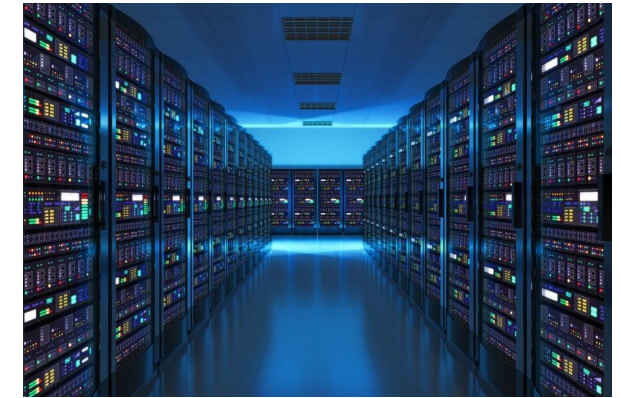


(2) Allow the use of libraries in other languages

MATSim
Multi-Agent Transport Simulation



(3) Use of computational power



Research question

How can large-scale agent-based simulations call expensive external services at runtime without losing scalability?

Why is this relevant?

- Public transport disturbances
- Evacuation scenarios
- On-demand services

Showcase

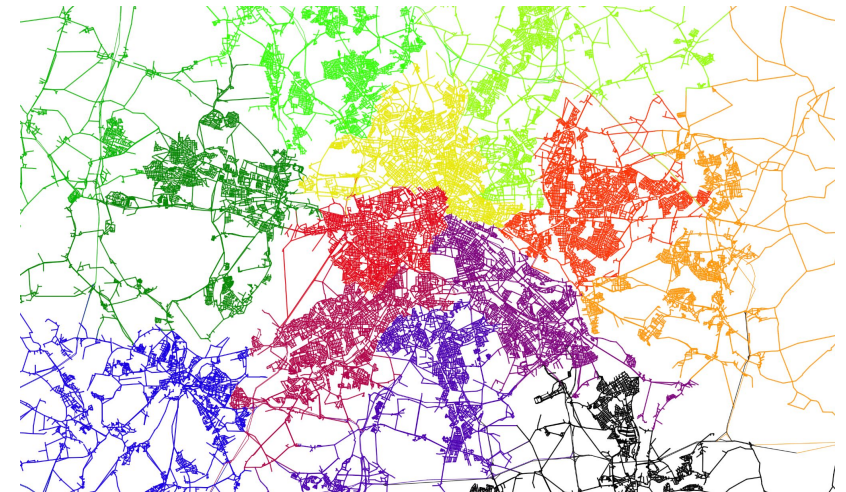
Computational Setup

- Single iteration in MATSim Rust
- Model: Berlin v6.4 (100% = 5mio agents)

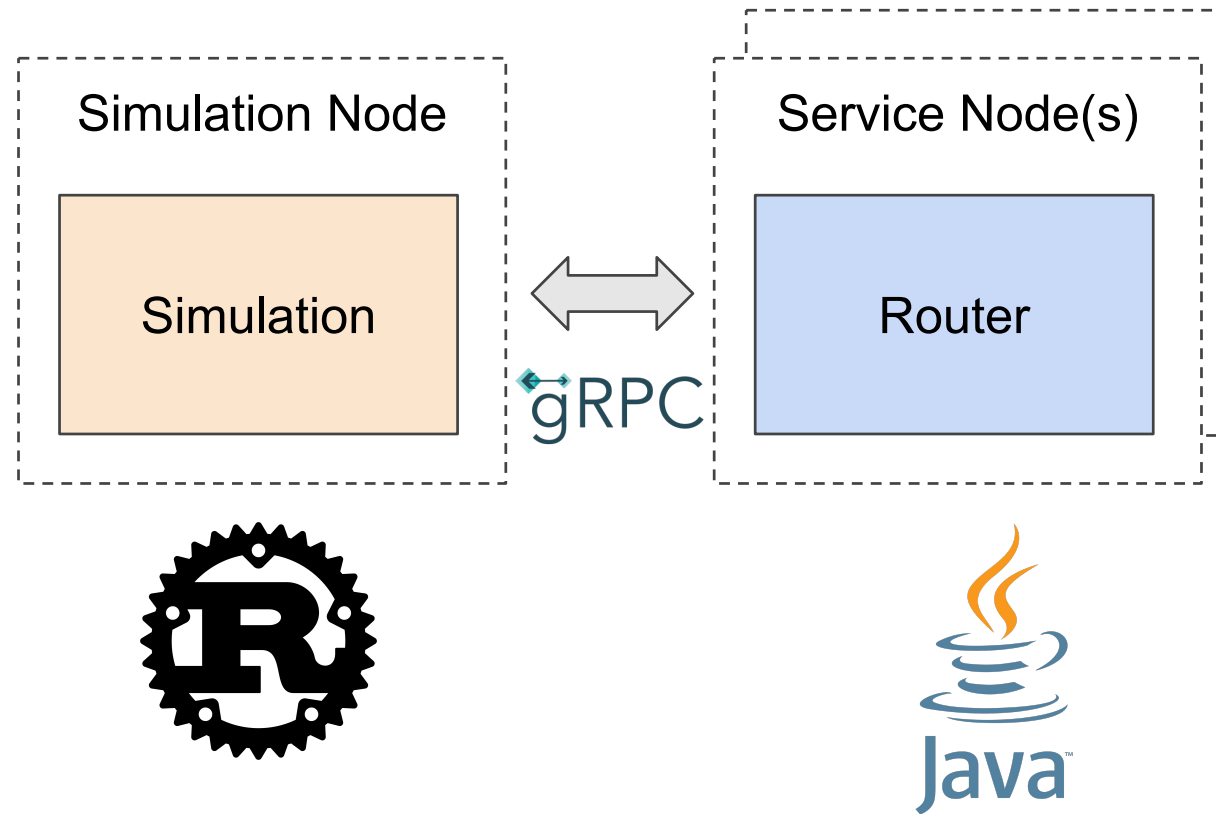


Demand Setup

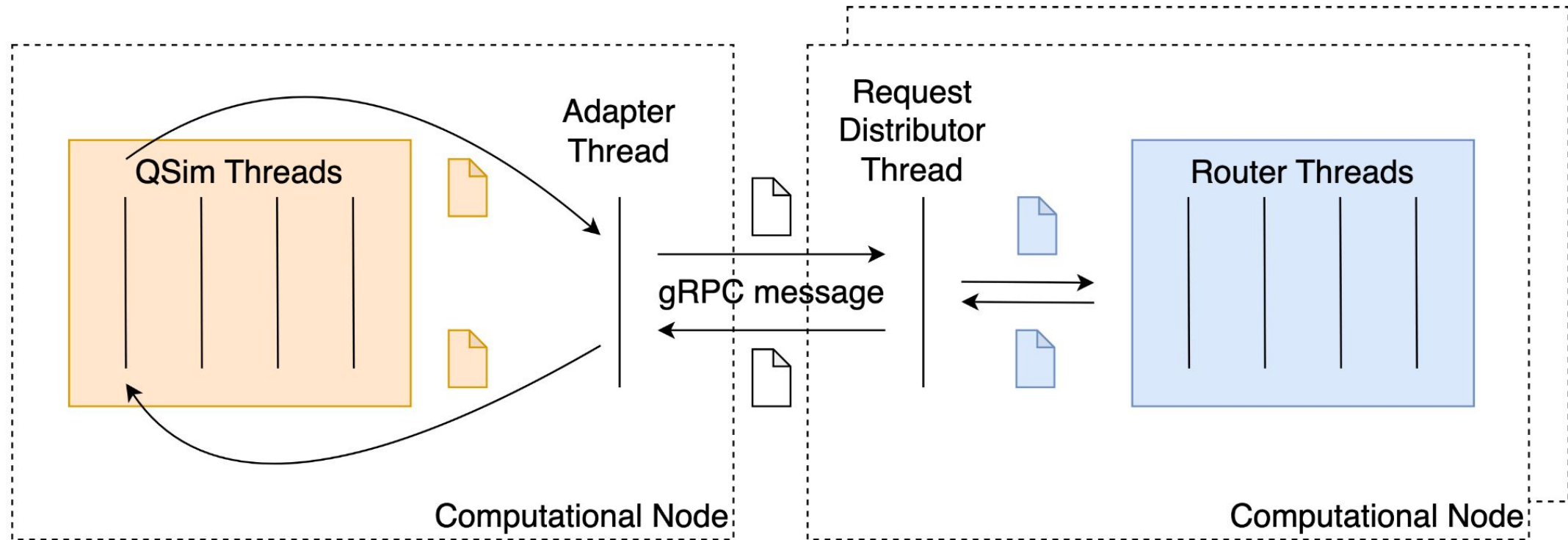
- Agents start with precomputed routes except for PT
- PT routes are calculated on the fly by an external service



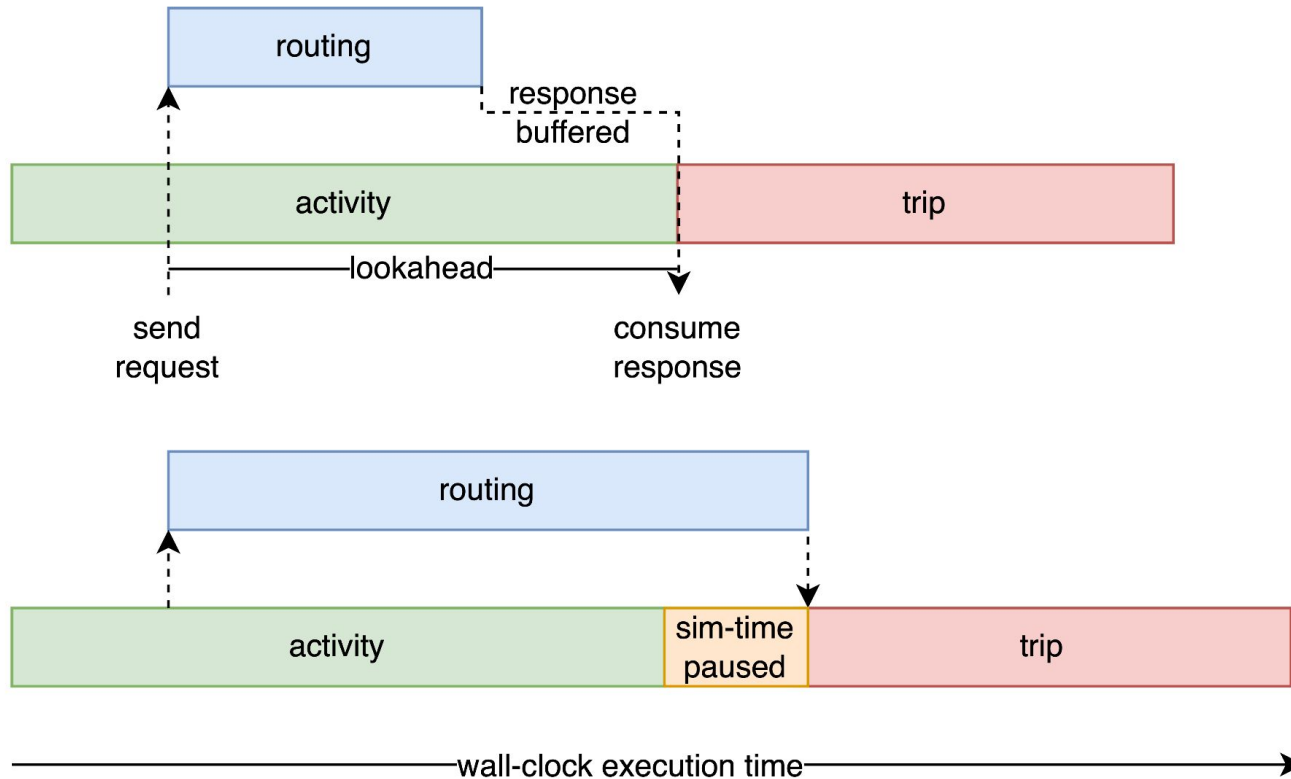
Architecture



Architecture



Core idea: use lookahead to hide routing latency

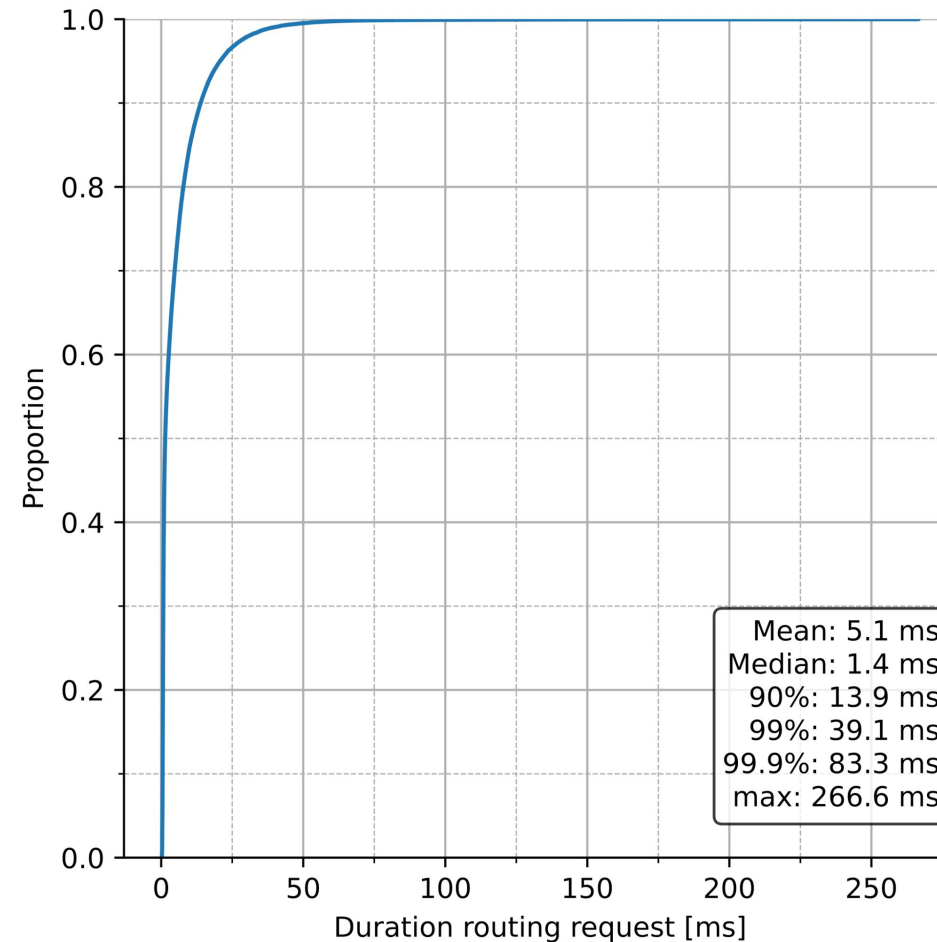


routing is fast → no pause



routing is slow → pause

Cumulative distribution of routing durations



Setting:

Berlin v6.4, 1% sample, 192 routing threads on 8 computational nodes (→ 24 threads/node)

Execution results

partitions	sample (%)	lookahead (sim seconds)	routing threads	RTR
192	1	0	1	601
192	1	0	192	754
192	1	200	1	621
192	1	200	192	7688
192	1	600	1	623
192	1	600	192	11684
192	1	1800	1	612
192	1	1800	192	12171
192	10	0	24	243
192	10	0	288	231
192	10	200	24	973
192	10	200	288	4579
192	10	600	24	1177
192	10	600	288	5016
192	10	1800	24	1250
192	10	1800	288	4568

x20

x20

RTR 12.171 ≈ 0.08ms/step
lookahead 1800 steps ≈ 147ms

$$\text{RTR} = \frac{\text{\#simulation seconds}}{\text{\#wall-clock seconds}}$$

Execution results

partitions	sample (%)	lookahead (sim seconds)	routing threads	RTR
192	1	0	1	601
192	1	0	192	754
192	1	200	1	621
192	1	200	192	7688
192	1	600	1	623
192	1	600	192	11684
192	1	1800	1	612
192	1	1800	192	12171
192	10	0	24	243
192	10	0	288	231
192	10	200	24	973
192	10	200	288	4579
192	10	600	24	1177
192	10	600	288	5016
192	10	1800	24	1250
192	10	1800	288	4568

Findings

- Lookahead itself**
boosts performance
- #routing threads** has
a great impact
- Smaller samples**
reach higher RTR

$$\text{RTR} = \frac{\text{\#simulation seconds}}{\text{\#wall-clock seconds}}$$

Execution results

partitions	sample (%)	lookahead (sim seconds)	routing threads	RTR
192	1	0	1	601
192	1	0	192	754
192	1	200	1	621
192	1	200	192	7688
192	1	600	1	623
192	1	600	192	11684
192	1	1800	1	612
192	1	1800	192	12171
192	10	0	24	243
192	10	0	288	231
192	10	200	24	973
192	10	200	288	4579
192	10	600	24	1177
192	10	600	288	5016
192	10	1800	24	1250
192	10	1800	288	4568

Findings

1. **Lookahead itself**
boosts performance
2. **#routing threads** has
a great impact
3. **Smaller samples**
reach higher RTR

$$\text{RTR} = \frac{\text{\#simulation seconds}}{\text{\#wall-clock seconds}}$$

Execution results

partitions	sample (%)	lookahead (sim seconds)	routing threads	RTR
192	1	0	1	601
192	1	0	192	754
192	1	200	1	621
192	1	200	192	7688
192	1	600	1	623
192	1	600	192	11684
192	1	1800	1	612
192	1	1800	192	12171
192	10	0	24	243
192	10	0	288	231
192	10	200	24	973
192	10	200	288	4579
192	10	600	24	1177
192	10	600	288	5016
192	10	1800	24	1250
192	10	1800	288	4568

Findings

1. **Lookahead itself**
boosts performance
2. **#routing threads** has
a great impact
3. **Smaller samples**
reach higher RTR

$$\text{RTR} = \frac{\text{\#simulation seconds}}{\text{\#wall-clock seconds}}$$

Take-away

Async execution hides service latency for $\geq 99\%$ of runtime routing calls.
This yields a 20× speedup over synchronous routing in Berlin scenarios.

Open question: How far can this scale for short activities and long-running routing requests?

MATSim Rust

Check out the GitHub repository:

<https://github.com/matsim-vsp/matsim-rust>

Lightweight and fast prototype of MATSim written in Rust
(implements route choice)

🔧 Contributions very welcome!

heinrich@vsp.tu-berlin.de

GitHub repository:



MATSim  **rust**